

Deep Learning

Working with Keras: A Deep Dive



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand how to build models using sequential API.
2. Understand how to build models using functional API.



Roadmap of this lecture:

1. Sequential Model

2. Functional API

2.1 Single-input Single-output model

2.2 Multiple-input multiple-output model

2.3 Reuse layer or model



Different ways to build Keras models

Sequential model: a stack of layers

Functional API: a graph-like model

Model subclassing: most flexible, low-level coding

Sequential Model

Sequential model

Listing 7.1 The Sequential class

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

Note that it's possible to build the same model incrementally via the `add()` method, which is similar to the `append()` method of a Python list.

Listing 7.2 Incrementally building a Sequential model

```
model = keras.Sequential()
model.add(layers.Dense(64, activation="relu"))
model.add(layers.Dense(10, activation="softmax"))
model.build((None, 3))
```



Sequential model

After the model is built, you can display its contents via the `summary()` method, which comes in handy for debugging.

Listing 7.5 The `summary()` method

```
>>> model.summary()  
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
=====		
dense_2 (Dense)	(None, 64)	256

dense_3 (Dense)	(None, 10)	650
=====		
Total params: 906		
Trainable params: 906		
Non-trainable params: 0		

Sequential model: **Name** model and even layers

Listing 7.6 Naming models and layers with the name argument

```
>>> model = keras.Sequential(name="my_example_model")
>>> model.add(layers.Dense(64, activation="relu", name="my_first_layer"))
>>> model.add(layers.Dense(10, activation="softmax", name="my_last_layer"))
>>> model.build((None, 3))
>>> model.summary()
Model: "my_example_model"
```

Layer (type)	Output Shape	Param #
=====		
my_first_layer (Dense)	(None, 64)	256

my_last_layer (Dense)	(None, 10)	650
=====		
Total params: 906		
Trainable params: 906		
Non-trainable params: 0		

Sequential model: Specify input size in advance

Listing 7.7 Specifying the input shape of your model in advance

```
model = keras.Sequential()  
model.add(keras.Input(shape=(3,)))  
model.add(layers.Dense(64, activation="relu"))
```

Use Input to declare the shape of the inputs. Note that the shape argument must be the shape of each sample, not the shape of one batch.

Quiz questions:

1. How to build a sequential model?
2. Can all models be built using sequential API?

Roadmap of this lecture:

1. Sequential Model

2. Functional API

2.1 Single-input Single-output model

2.2 Multiple-input multiple-output model

2.3 Reuse layer or model



Functional API

Listing 7.8 A simple Functional model with two Dense layers

```
inputs = keras.Input(shape=(3,), name="my_input")
features = layers.Dense(64, activation="relu")(inputs)
outputs = layers.Dense(10, activation="softmax")(features)
model = keras.Model(inputs=inputs, outputs=outputs)
```


Functional API

Listing 7.8 A simple Functional model with two Dense layers

```
inputs = keras.Input(shape=(3,), name="my_input")
features = layers.Dense(64, activation="relu")(inputs)
outputs = layers.Dense(10, activation="softmax")(features)
model = keras.Model(inputs=inputs, outputs=outputs)
```

inputs

```
>>> inputs.shape
(None, 3)
>>> inputs.dtype
float32
```

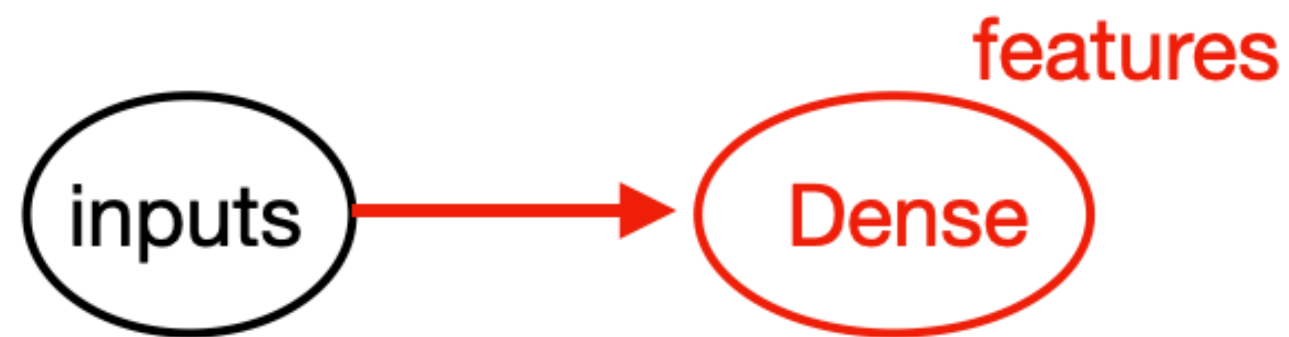
The model will process batches where each sample has shape (3,). The number of samples per batch is variable (indicated by the None batch size).

These batches will have dtype float32.

Functional API

Listing 7.8 A simple Functional model with two Dense layers

```
inputs = keras.Input(shape=(3,), name="my_input")
features = layers.Dense(64, activation="relu")(inputs)
outputs = layers.Dense(10, activation="softmax")(features)
model = keras.Model(inputs=inputs, outputs=outputs)
```

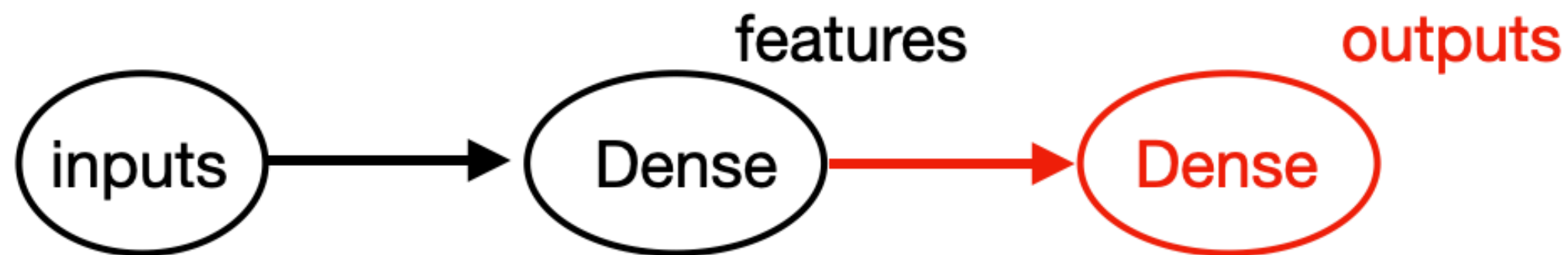


```
>>> features.shape
(None, 64)
```


Functional API

Listing 7.8 A simple Functional model with two Dense layers

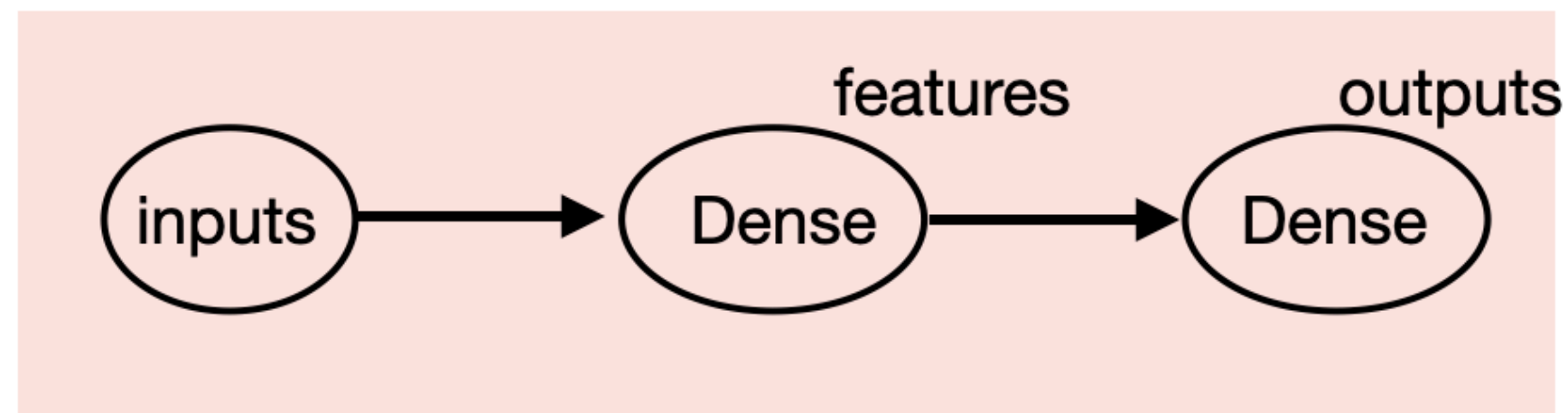
```
inputs = keras.Input(shape=(3,), name="my_input")
features = layers.Dense(64, activation="relu")(inputs)
outputs = layers.Dense(10, activation="softmax")(features)
model = keras.Model(inputs=inputs, outputs=outputs)
```



Functional API

Listing 7.8 A simple Functional model with two Dense layers

```
inputs = keras.Input(shape=(3,), name="my_input")
features = layers.Dense(64, activation="relu")(inputs)
outputs = layers.Dense(10, activation="softmax")(features)
model = keras.Model(inputs=inputs, outputs=outputs)
```



model

```
>>> model.summary()
Model: "functional_1"
```

Layer (type)	Output Shape	Param #
=====		
my_input (InputLayer)	[(None, 3)]	0

dense_6 (Dense)	(None, 64)	256

dense_7 (Dense)	(None, 10)	650
=====		
Total params: 906		
Trainable params: 906		
Non-trainable params: 0		

Quiz questions:

1. How to build a model using functional API?
2. Can all models be built using functional API?

Roadmap of this lecture:

1. Sequential Model

2. Functional API

2.1 Single-input Single-output model

2.2 Multiple-input multiple-output model

2.3 Reuse layer or model



Functional API: **Multi-Input, Multi-Output Model**

Let's say you're building a system to rank customer support tickets by priority and route them to the appropriate department. Your model has three inputs:

- The title of the ticket (text input)
- The text body of the ticket (text input)
- Any tags added by the user (categorical input, assumed here to be one-hot encoded)

Functional API: **Multi-Input, Multi-Output Model**

Your model also has two outputs:

- The priority score of the ticket, a scalar between 0 and 1 (sigmoid output)
- The department that should handle the ticket (a softmax over the set of departments)

Listing 7.9 A multi-input, multi-output Functional model

```
vocabulary_size = 10000
num_tags = 100
num_departments = 4
```

**Define
model
inputs.**

```
title = keras.Input(shape=(vocabulary_size,), name="title")
text_body = keras.Input(shape=(vocabulary_size,), name="text_body")
tags = keras.Input(shape=(num_tags,), name="tags")
```

**Combine input features into
a single tensor, features, by
concatenating them.**

```
features = layers.Concatenate()([title, text_body, tags])
features = layers.Dense(64, activation="relu")(features)
```

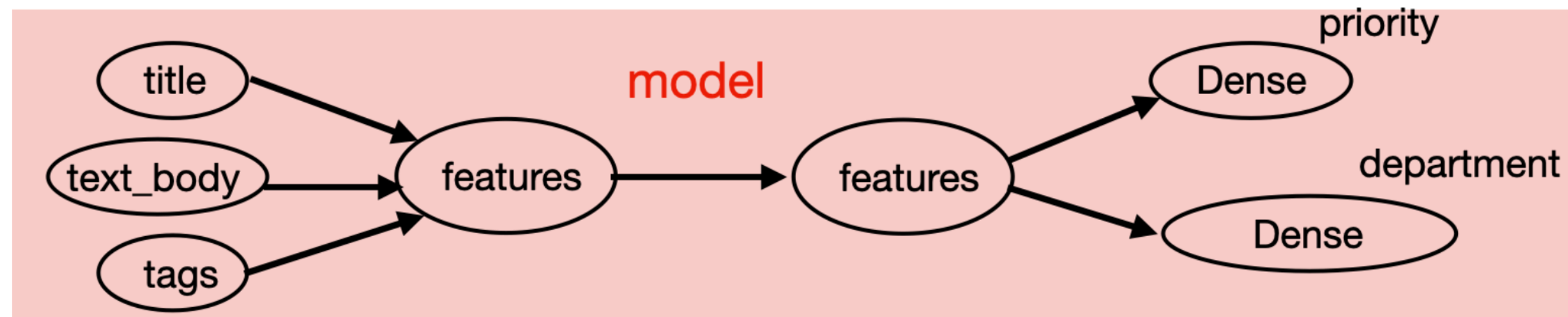
**Define
model
outputs.**

```
priority = layers.Dense(1, activation="sigmoid", name="priority")(features)
department = layers.Dense(
    num_departments, activation="softmax", name="department")(features)
```

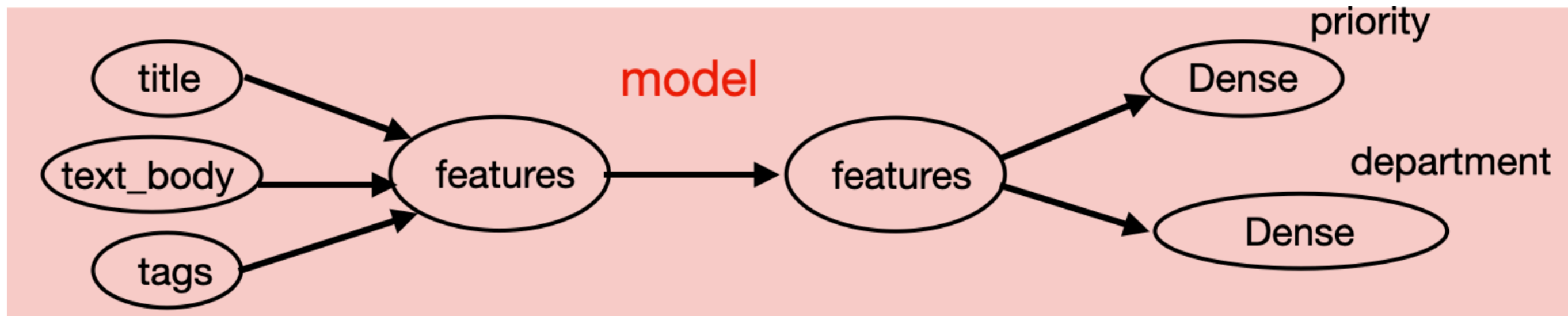
```
model = keras.Model(inputs=[title, text_body, tags],
                    outputs=[priority, department])
```

Create the model by specifying its inputs and outputs.

**Apply an intermediate
layer to recombine input
features into richer
representations.**



```
model.compile(optimizer="rmsprop",
              loss=["mean_squared_error", "categorical_crossentropy"],
              metrics=[["mean_absolute_error"], ["accuracy"]])
model.fit([title_data, text_body_data, tags_data],
        [priority_data, department_data],
        epochs=1)
model.evaluate([title_data, text_body_data, tags_data],
              [priority_data, department_data])
priority_preds, department_preds = model.predict(
    [title_data, text_body_data, tags_data])
```



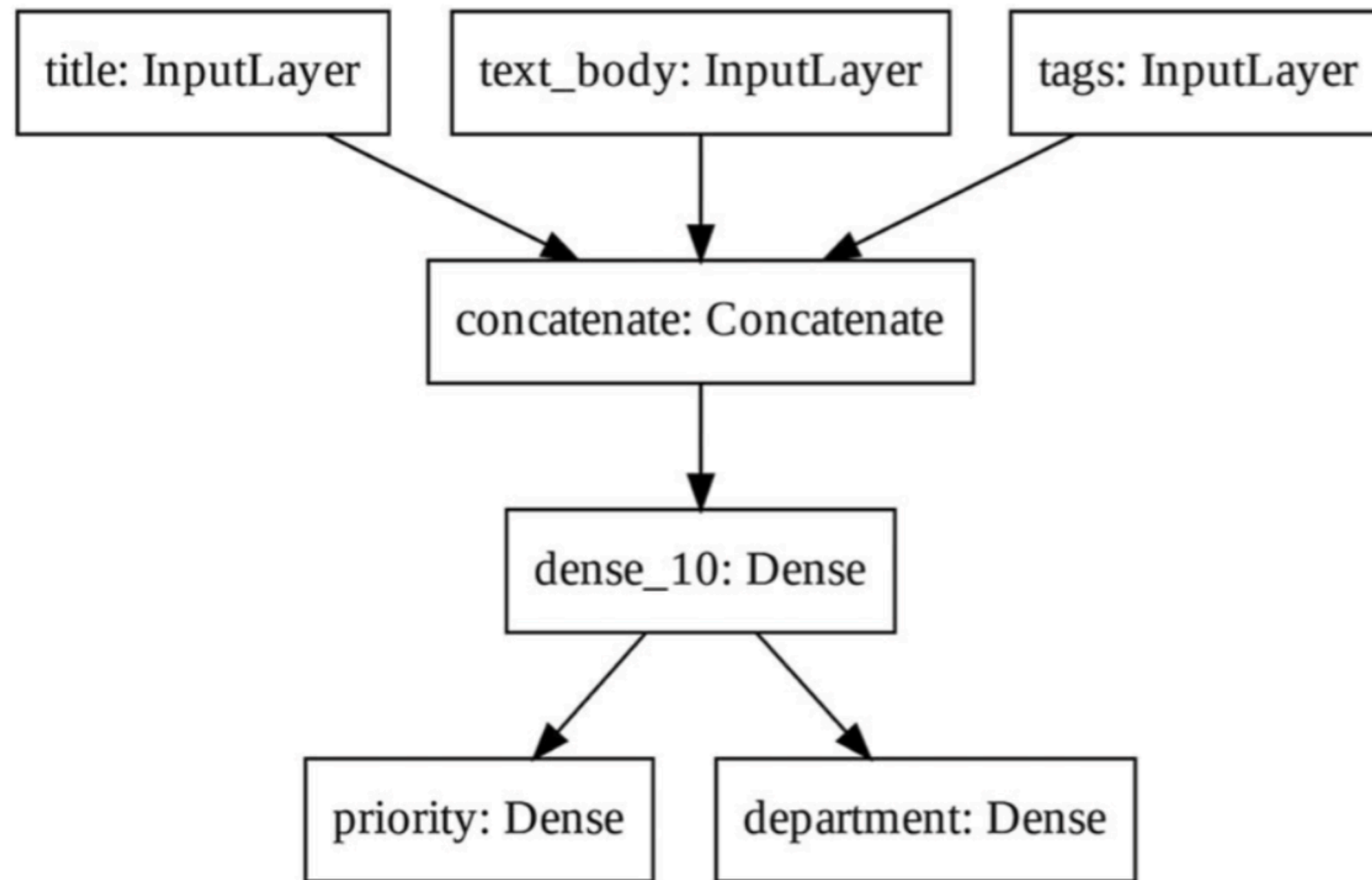
If you don't want to rely on input order (for instance, because you have many inputs or outputs), you can also leverage the names you gave to the Input objects and the output layers, and pass data via dictionaries.

Listing 7.11 Training a model by providing dicts of input and target arrays

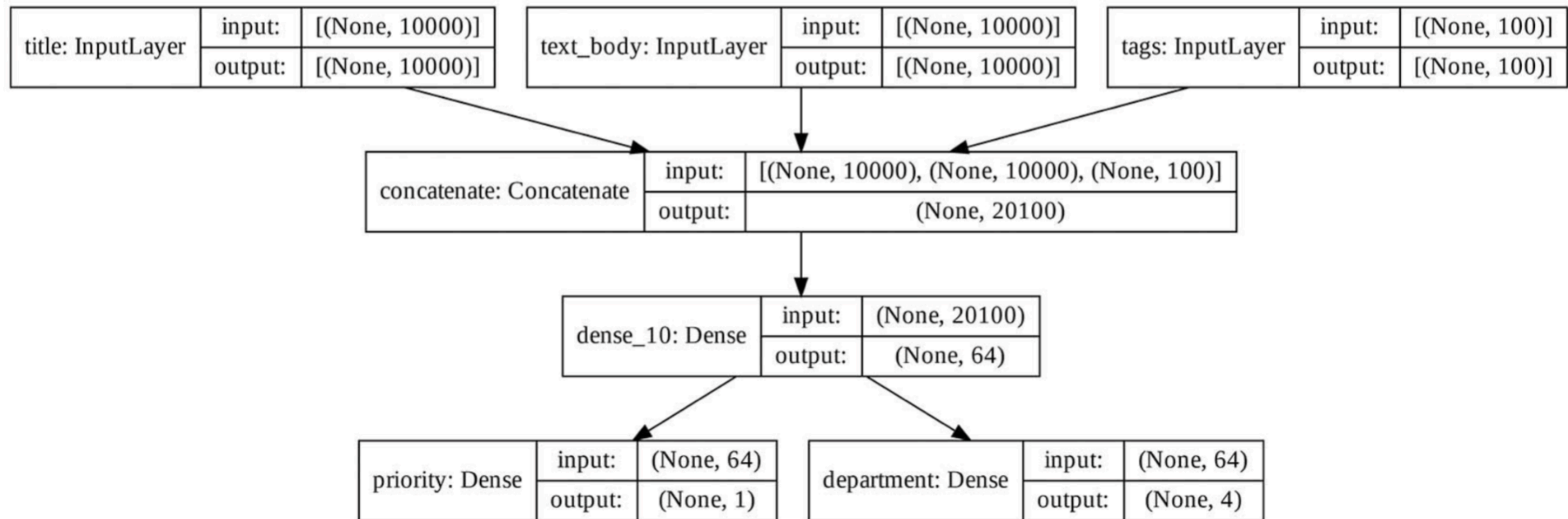
```
model.compile(optimizer="rmsprop",
              loss={"priority": "mean_squared_error", "department":
                  "categorical_crossentropy"},
              metrics={"priority": ["mean_absolute_error"], "department":
                      ["accuracy"]})
model.fit({"title": title_data, "text_body": text_body_data,
         "tags": tags_data},
        {"priority": priority_data, "department": department_data},
        epochs=1)
model.evaluate({"title": title_data, "text_body": text_body_data,
               "tags": tags_data},
               {"priority": priority_data, "department": department_data})
priority_preds, department_preds = model.predict(
    {"title": title_data, "text_body": text_body_data, "tags": tags_data})
```


Visualize Model's Topology

```
keras.utils.plot_model(model, "ticket_classifier.png")
```



```
keras.utils.plot_model(
    model, "ticket_classifier_with_shape_info.png", show_shapes=True)
```



Quiz questions:

1. How to build a multi-input multi-output model?
2. Why are multi-input multi-output models useful?

Roadmap of this lecture:

1. Sequential Model

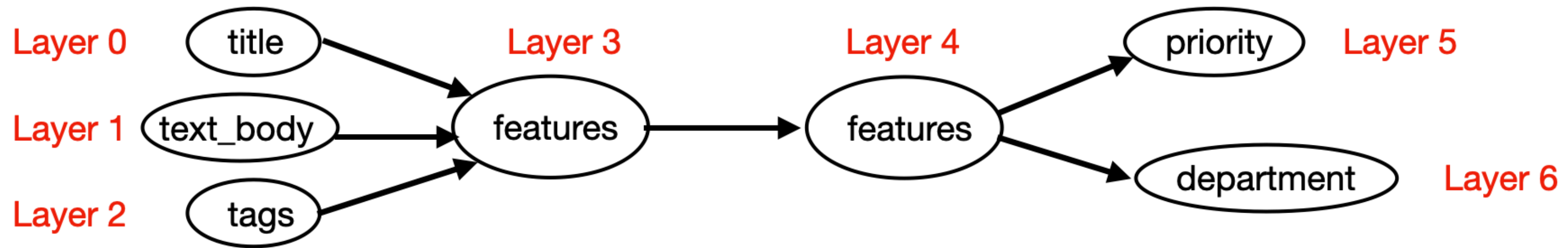
2. Functional API

2.1 Single-input Single-output model

2.2 Multiple-input multiple-output model

2.3 Reuse layer or model





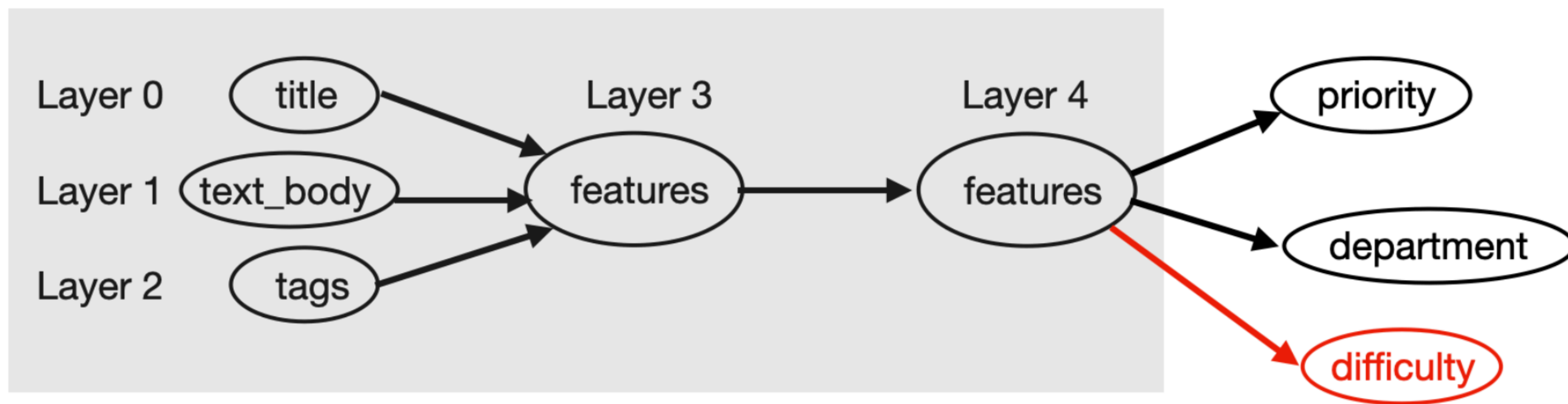
Let's say you want to add another output to the previous model—you want to estimate how long a given issue ticket will take to resolve, a kind of difficulty rating. You could do this via a classification layer over three categories: “quick,” “medium,” and “difficult.” You don't need to recreate and retrain a model from scratch. You can start from the intermediate features of your previous model, since you have access to them, like this.

Listing 7.13 Creating a new model by reusing intermediate layer outputs

```
features = model.layers[4].output
difficulty = layers.Dense(3, activation="softmax", name="difficulty")(features)

new_model = keras.Model(
    inputs=[title, text_body, tags],
    outputs=[priority, department, difficulty])
```

← layers[4] is our intermediate
Dense layer



Quiz questions:

1. How to reuse a layer or a model?
2. When is it useful to reuse a layer or a model?